



نظري

5

12

1440 sp

كلية الهندسة المعلوماتية

السنة الثانية

العمليات الأساسية على السجلات 2

د. سيرا أستور



RB Informatcs; 05/05/2025

محتوى مجاني غير مخصص للبيع التجاري

بنیان الحاسوب 1

بسم الله الرحمن الرحيم

تذكرة:

معالج MIPS يُعد من معالجات load/store أي أن التعامل مع الذاكرة يتم فقط من خلال تعليمات التحميل load والتخزين store، فلا يسمح لتعليمات مثل الجمع أو الطرح بالعمل مباشرة على البيانات الموجودة في الذاكرة، بل يجب أولاً تحميل البيانات إلى السجلات، ثم تنفيذ العمليات عليها، وبعدها تخزين النتائج مرة أخرى في الذاكرة.

عند تصميم أي معالج يتم اختيار عدة طرق عنوانة Addressing Modes لتحديد كيفية الوصول إلى البيانات في الذاكرة، في MIPS الطريقة الأساسية والأكثر استخداماً هي طريقة Base + offset وتستخدم مع التعليمات مثل lw (load word) و sw (store word).

شرح طريقة Base + offset:

- يتم استخدام سجل (Base Register) يحتوي على عنوان أساسي.
- ويتم إضافة إزاحة ثابتة offset إلى هذا العنوان.
- الإزاحة يمكن أن تكون موجبة (للأمام في الذاكرة) أو سالبة (للخلف في الذاكرة)

Example:

lw \$t0, 4(\$s1)

يتم تحميل كلمة من الذاكرة عند الموقع $4 + \$s1$ إلى $\$t0$.

ملاحظة:

- الذاكرة تعنون بالـ Byte: يعني أن العناوين تزداد 1, 1, 1... (لكل Byte).
- MIPS يعالج البيانات بالـ Word: يعني أن التعليمات مثل lw, sw تتعامل دائماً مع 4 byte دفعة واحدة، وتطلب عنوان يبدأ من مضاعفات الـ 4: (0, 4, 8, 12, 16, ...).

وهذا سبب وجود ما يسمى بـ Alignment Restrictions.

- لا يسمح بأن تكون الكلمة (word) في عنوان ليس مضاعفاً لـ 4
- يعني مثلاً لا يمكن تنفيذ lw من عنوان 5 أو 6، بل فقط 0, 4, 8, ...
- بالتالي عند التعامل مع الـ word (مثل استعمال lw / sw / ..) يجب أن يكون العنوان من مضاعفات العدد 4 أي آخر 2 bits تحملان القيمة 0 دوماً.

Example:

0100=4

1000=8

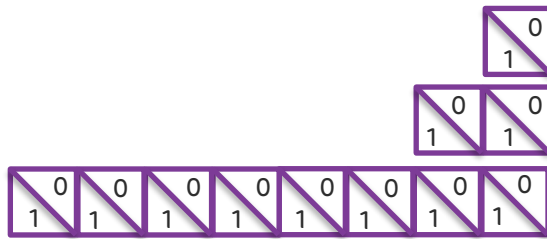
1100=12

ملاحظة: أما عند التعامل مع بايت (مثل استعمال sb / lb / ..) ف يسمح ب استخدام أي عنوان.

فكر؟!

كم Bit نحتاج لعنونة 4 Byte (أي عنونة 4 مواقع)؟

نحتاج إلى 2 Bit فقط.



■ لأن 1 bit عدد العناوين الممكنة 2.

■ لأن 2 bit عدد العناوين الممكنة 4.

■ لأن 3 bit عدد العناوين الممكنة 8.

و هكذا...

لكل bit احتمالين (0,1)

كيف يتم الجمع بين قيمة 16 بت وقيمة 32 بت؟

Example:

addi \$t0, \$t1, 10

إذا أردنا جمع عدد 16 بت مع عدد 32 بت (موجود في السجل) يجب أن نمدد الـ 16 بت إلى 32 بت.

كيف يتم التمديد؟

التمديد مع إشارة (SXT) Sign Extension:

يستخدم عندما يكون الرقم ذو إشارة (signed)، نأخذ البت الأخير (بت الإشارة) من 16 بت ونعيد نسخه حتى نملأ البتات العليا.

Example:

0000 0000 0000 1010 → 10

بت الإشارة هو (0) لذا نملأ بـ (0) حتى نصل لـ 32 بت.

0000 0000 0000 0000 0000 0000 1010 → 10

وهذا التمديد يتم تلقائياً داخل المعالج قبل تنفيذ العملية.

■ لاحظ:

lw rt, immediate(rs) rt → outputتحميل/قراءة word من الذاكرة إلى السجل (rt).sw rt, immediate(rs) rt → inputتخزين word من سجل (rt) إلى الذاكرة.

Translating MIPS Assembly Language into Machine Language

لدينا مصفوفة A مخزنة في الذاكرة وقاعدتها (عنوان أول عنصر فيها) هو \$t1، ولدينا المتغير h موجود في السجل \$s2، نريد تنفيذ التعليمة التالية: $A[300] = h + A[300]$

1. الوصول إلى A[300] في الذاكرة:

- كل عنصر في A هو 4 byte .int
- إذن: عنوان الـ byte الأول من العنصر رقم 300 هو: $300 \times 4 = 1200$.
- أي A[300] موجود في العنوان \$t1+1200
- تحميل A[300] من الذاكرة إلى \$t0

lw \$t0, 1200(\$t1)

2. جمع h + A[300]:

add \$t0, \$s2, \$t0

3. تخزين الناتج مرة ثانية في A[300] (أي في الذاكرة):

sw \$t0, 1200(\$t1)

الحل:

lw \$t0, 1200(\$t1)

add \$t0, \$s2, \$t0

sw \$t0, 1200(\$t1)

حول التعليمات السابقة إلى machine language:

(يذكر في نص المسألة الـ op وإذا كانت التعليمة من الـ R-type يتم ذكر الـ funct أيضاً)

Instruction	Format	op
add	R	0
sub (subtract)	R	0
add immediate	I	8 _{ten}
lw (load word)	I	35 _{ten}
sw (store word)	I	43 _{ten}

\$t0 = 8, \$t1 = 9, \$s1 = 18.

op	rs	rt	rd	shamt/ address	funct
35	9	8		1200	
0	18	8	8	0	32
43	9	8		1200	

100011	01001	0100	0000 0100 1011 0000		
000000	10010	0100	0100	0000	100000
101011	01001	0100	0000 0100 1011 0000		

شرح أربيسي:

شرح السطر الأول من كلا الجدولين (أي التعليمة `(lw $t0, 1200($t1))`:

الـ op معطى في الجدول السابق ← 35

$rs = \$t1 = 9, rt = \$t0 = 8$ (أرقام السجلات حفظ)

الـ address هو 1200.

وفي الجدول الثاني يتم تحويل الأرقام إلى النظام الثنائي.

"Don't watch the clock;
do what it does, keep going."



Signed and unsigned loads in MIPS:

في MIPS (لغة تجميع Assembly) يوجد نوعان من الأعداد: Signed (ذات إشارة) و unsigned (غير ذات إشارة)، وهي أعداد صحيحة لا تحوي فواصل عشرية.

- Signed: يمكن أن تكون موجبة أو سالبة (-3, 0, 5, ...).
- Unsigned: يمكن أن تكون موجبة فقط (0, 3, 100, ...).

التحميل من الذاكرة (load):

لدينا طريقتين:

1. lb (load byte):

يحمل الـ byte ويفترض أنه signed.

يقوم بتوسيع الإشارة (signed extension) أي يملأ البتات المتبقية بنفس الإشارة (1 للسالب، 0 للموجب).

2. lbu (load byte unsigned):

يحمل الـ byte ويعامله كـ unsigned.

لا يملأ الإشارة، فقط يضيف أصفار إلى اليسار لتكملة الـ 32 بت.

استخدام الـ unsigned:

أحياناً نحتاج إلى تمثيل أرقام لا يمكن أن تكون سالبة مثل الـ memory addresses.
في لغة C يكون:

```
int x;           // عدد signed
unsigned int y;  // عدد unsigned
```

■ ملاحظة:

يتم استخدام lb مع المحارف (Characters)، لأن المحرف في البرمجة يمثل باستخدام Byte واحد وتعليمة lb هي "load byte" أي تحميل Byte واحد فقط من الذاكرة إلى السجل.

MIPS Branch Instructions

وهي تعليمات تستخدم لاتخاذ القرارات (Decision making):

beq (branch if equal):	اذهب إلى مكان آخر إذا كانت القيمتان متساويتان
bne (branch if not equal):	اذهب إلى مكان آخر إذا كانت القيمتان غير متساويتان

I-type	op CODE	rs	rt	16-bit signed constant
--------	---------	----	----	------------------------

كيف تعمل؟

if (Reg [rs] == Reg [rt]) => pc ← (pc+4) + 4*sxt(immediate)

if (Reg [rs] != Reg [rt]) => pc ← (pc+4) + 4*sxt(immediate)

If we don't take the branch: (condition is false)

$$PC = PC + 4.$$

If we do take the branch: (condition is true)

$$PC = (PC + 4) + (\text{immediate} * 4).$$

Example:

Assuming (i at \$s0, j at \$s1, h at \$s3)

In C	In assembly language
if(i == j) h=i + j;	bne \$s0, \$s1, label add \$s3, \$s0, \$s1 label:

تفصيل المعادلة:

$$pc \leftarrow (pc+4) + 4*sxt(immediate)$$

تذكرة:

pc هو السجل الذي يحوي على عنوان التعليمة التالية، بعد تنفيذ كل تعليمة MIPS بشكل طبيعي يزيد pc بمقدار 4 (لأن كل تعليمة حجمها 4 byte).

1. (pc+4):

هذا هو العنوان الافتراضي للتعليمة التالية، لأن المعالج دائماً ينتقل للتعليمة التالية بعد تنفيذ الحالية.

2. sxt (Immediate):

هي عدد الخطوات (التعليمات) التي نريد القفز بها (موجبة أو سالبة)، نضرب هذه القيمة بـ 4 لأن العناوين في MIPS تتحرك بخطوات 4 byte (حجم كل تعليمة).

وهذا يعطينا الإزاحة الفعلية التي سنقفز بها، يعني أننا نحسب:

العنوان الجديد = عنوان التعليمة التالية + الإزاحة.

ملاحظة:

نقوم بعمل sign Extension لأن هذه القيمة داخل التعليمة تكون بحجم 16 بت فقط بينما عنوان الذاكرة (pc) في MIPS هو 32 بت.

لذلك: يجب أن نمدد هذه القيمة لتصبح 32 بت حتى نستطيع جمعها مع الـ PC ونحسب العنوان الجديد.

■ ملاحظة:

في معمارية MIPS، التعليمات الحقيقية الوحيدة لاتخاذ القرار هي مثل beq و bne أي مقارنات أخرى مثل "أكبر من" أو "أصغر من" ما هي إلا تعليمات زائفة (pseudo-instruction) يقوم الـ Assembler بتحويلها إلى تعليمات MIPS حقيقية ثم تحول هذه إلى أصفار ووحدات.

Branch Relative Addressing

في التعليمات نستخدم immediate لتحديد مكان القفز لكن الـ Immediate حجمه 16 بت فقط، أي لا يمكننا كتابة عنوان كامل (32 بت) في التعليمة، وبالتالي لا يمكن القفز لأي مكان في الذاكرة، فقط أماكن قريبة من مكاننا الحالي (pc).
الحل:

PC-Relative Addressing

بدلاً من كتابة عنوان كامل نحدد كم تعليمة نريد القفز إليها نسبةً إلى الموقع الحالي (pc).

16 bits تعطينا مجال قفز $2^{15} \pm$ عن الـ pc، لكن في الحقيقة التعليمات هي words وكل word هي 4 byte، يعني كل قيمة immediate يتم ضربها بـ 4 أي نستطيع أن نقفز 4×2^{15} أي $2^{17} \pm$ وهذا يسمح لنا بكتابة loops أكبر بكثير تصل لـ 4 أضعاف الحجم الذي كنا نستطيعه لو كنا نقفز بالبايت دون مضاعفة $\times 4$.

Control Flow:

لدينا beq, bne لكن ماذا عن "Branch-if-less than"؟
تعليمية (blt) Branch-if-less than غير موجودة خطأً في تعليمات MIPS الأصلية، لكن يمكننا أن نحاكيها باستخدام تعليمات موجودة، وذلك باستخدام bne, slt.

■ slt (set if less than):
`slt $t0, $s1, $s2` => if $\$s1 < \$s2$
 then $\$t0=1$
 else
 $\$t0=0$

ثم نستخدم bne بعدها:

`bne $t0, $zero, label`

أي إذا كانت $\$t0 \neq \0 والشرط محقق إذاً اذهب إلى label.

■ ملاحظة:

إذا كان $\$s1 < \$s2$ نضع slt قيمة 1 في $\$t0$ ثم إذا كان $\$t0 \neq \0 (أي $\$t0=1$ أي الشرط محقق) نقفز في bne إلى label.

```
slt $1, $s1, $s2
bne $1, $zero, label } blt $s1, $s2, label
```

ويمكن استعمال blt مباشرة كونها تعليمة زائفة pseudo-instruction.
أي أنّ تعليمات المقارنة هي تعليمات مزيفة تترجم لتعليمتين حقيقتين معاً:
الأولى تعليمة حسابية Arithmetic Instruction وهي slt.
والثانية تعليمة تحكم Control Instruction وهي bne.

Singed vs. Unsigned comparison

Example:

لدينا محتوى السجل \$s0 هو: 1111 1111 1111 1111 1111 1111 1111
هذا يعني:

ك signed القيمة هي (-1).

ك unsigned القيمة هي (4,294,967,295).

ولدينا محتوى السجل \$s1 هو: 0000 0000 0000 0000 0000 0000 0001

سواء signed أو unsigned القيمة هي (1).

ماهي قيمة \$t0 و \$t1 بعد كلتا التعليمتين؟

1. slt \$t0, \$s0, \$s1

هذه مقارنة موقعة signed، أي هل $-1 > 1$ ؟ الجواب نعم، لذا $t0=1$.

2. sltu \$t1, \$s0, \$s1

هذه مقارنة غير موقعة unsigned، أي هل $4,294,967,295 > 1$ ؟ الجواب لا، لذا $t1=0$.

4 combinations of slt & (beq / bne):

slt \$t0, \$s0, \$s1 bne \$t0, \$0, Less	# \$t0 = 1 if g<h # if (g<h) goto Less
slt \$t0, \$s1, \$s0 bne \$t0, \$0, Grtr	# \$t0 = 1 if g>h # if (g>h) goto Grtr
slt \$t0, \$s0, \$s1 beq \$t0, \$0, Gteq	# \$t0 = 1 if g<h If (g>=h) goto Gteq
slt \$t0, \$s1, \$s0 beq \$t0, \$0, Lteq	# \$t0 = 1 if g>h # if (g<=h) goto Lteq

Pseudo Instructions:

blt \$rs, \$rt, label	Branch if less than
bgt \$rs, \$rt, label	Branch if Greater than
ble \$rs, \$rt, label	Branch if less or Equal
bge \$rs, art, label	Branch if Greater or Equal
la \$rd, address	Load Address
li \$rd, immediate	load Immediate
move \$rd, \$rs	نسخ قيمة من سجل لآخر

Example:

la \$s1, x

يعني حمّل عنوان المتغير x في السجل \$s1

li \$s1, x

يعني حمّل القيمة x نفسها (كبيانات) في السجل \$s1

لكن كيف يترجمهم الـ Assembler؟

إذا كانت القيمة أو العنوان 32 بت (كبيرة) يقوم الـ Assembler بتجزئتها إلى: **lui**=load upper immediate تحمل النصف العلوي من القيمة و: **ori**=OR immediate تضيف النصف السفلي (16 بت السفلي) إلى القيمة الموجودة.

lui \$s1, upper(x)

ori \$s1, \$s1 Lower(x)

ملاحظة أرسيزية:

التعليمات الزائفة لا تنتمي مباشرة لأي نوع (R/I/J) لكنها تترجم لتعليمات حقيقية وكل منها ينتمي لنوع معين.

MIPS Jumps

في MIPS تعليمات الـ branch مثل beq و bne عندها مدى محدود، يمكنها القفز فقط تقريباً $\pm 32,000$ تعليمة والسبب: أن الحقل immediate حجمه 16 بت (يعني 2^{15})، لذا للقفز لمسافات أبعد يمكن استخدام تعليمات القفز غير المشروط (Jump instructions).

1. label z:

تقفز مباشرة إلى عنوان معين، تحسب العنوان الجديد باستخدام:

$$pc = PC[31:28] \parallel (CONST[25:0] \ll 2)$$

pc [31:28]: نأخذ أعلى 4 بت من العنوان الحالي pc

CONST [25:0]: هو العنوان الذي كتب بعد label z، يتم حفظه بـ 26 بت في التعليمة.

$\ll 2$: نضرب بـ 4 (shift left by 2) لأن التعليمات في MIPS كلها بحجم 4 byte (word). نقوم بهذا لأن (z) لا تحمل العنوان الكامل (32 بت) بسبب حجمها المحدود.

2. jal label (Jump And Link)

نفس label z لكن أيضاً تخزن عنوان العودة (pc+4) في السجل \$ra الذي هو \$31.

تذكرة:

\$ra return address

ملاحظة أرسيزية:

تستخدم لاستدعاء دالة Function call

3. jr \$t0 (Jump Register)

تقفز إلى العنوان الموجود داخل السجل \$t0.

ملاحظة أرسيزية:

تستخدم عادة للعودة من دالة.

4. jalr \$t0, \$ra (Jump And Link Register)

- تقفز إلى عنوان موجود في السجل \$t0.
- تخزن عنوان العودة في سجل آخر \$ra.

ملاحظة هامة:

ليس كل الـ Architectures تدعم جميع هذه التعليمات.

وظيفة	التعليمة
قفز مباشر بعيد	j label
قفز مباشر + حفظ عنوان العودة	jal label
قفز إلى عنوان في سجل	jr \$reg
قفز إلى عنوان في سجل + حفظ عنوان العودة	jalr \$reg, \$ra

J-type: used for j	op = 2	26-bit constant				
J-type: used for jal	op = 3	26-bit constant				
R-type: used for jr	op = 0	rs	0	0	0	func=8
R-type: used for jalr	op = 0	rs	0	rd	0	func=9

■ MIPS Jump Example:

C code	MIPS code
If (i != j) h=i + j; else h=i-j;	beq \$s4, \$s5, label add \$s3, \$s4, \$s5 j lab2 lab1: sub \$s3, \$s4, \$s5 lab2:

حيث:

i => \$s4

j => \$s5

h => \$s3

Loops MIPS Code

Loop:

beq \$9, \$0, End

add \$8, \$8, \$10

addi \$9, \$9, -1

(العودة لبداية الحلقة) j Loop

End:

What is the code of beq instructions?

op of beq = 4 (look up in table)

rs = 9 (first operand)

rt = 0 (second operand)

immediate = ???

In this case, immediate = 3

لأنها تمثل عدد التعليمات التي يجب تجاوزها (القفز فوقها) أي الفرق بين beq و End وهي 3 تعليمات.

■ ملاحظة:

في تعليمات ال branching مثل beq، نكتب الإزاحة (offset) بعدد التعليمات (words) وليس بعدد البايتات، لأن المعالج يضربها تلقائياً في 4.

يرجى قراءة ما تبقى من سلايدات الدكتوراة (من سلايد 53 حتى نهاية ملف ISA)



انتهت المحاضرة